Державний університет «Житомирська політехніка»
Факультет інформаційно-комп’ютерних технологій
Кафедра комп’ютерної інженерії

ПОЯСНЮВАЛЬНА ЗАПИСКА

до лабораторної роботи № 2
з дисципліни «Програмування»

Розгалужені та циклічні обчислювальні процеси у мові С, консольні меню та оформлення інженерної звітності

Індивідуальний варіант 1

Виконав студент 1 курсу
групи КІ-26-1
Безвершук Ростислав

Перевірив Лисенко М. С.
Посада: Старший викладач кафедри комп'ютерної інженерії та штучного інтелекту
Науковий ступінь: Доктор філософії з інженерії програмного забезпечення

Житомир 2026

1 Мета роботи та індивідуальні завдання

Мета роботи - набути практичних навичок побудови розгалужених і циклічних алгоритмів мовою C, реалізувати чисельне обчислення функцій за степеневими рядами, перевіряти вхідні дані та оцінювати похибку результатів. Під час виконання роботи використовую GCC, термінал і Git для компіляції, перевірки та фіксації вихідних файлів.

Номер у списку академічної групи - 1. За правилом методичних вказівок отримую варіант 1:

$$V = ((1 - 1)\ mod\ 15) + 1 = 1$$

1.1 Завдання 1

Розробити task1.c для табулювання наведеної функції на відрізку [Xstart; Xend] із кроком dX. Межі та крок зчитати як double. Перевірити Xstart < Xend та dX > 0; при некоректних параметрах вивести повідомлення й завершити програму з ненульовим кодом.

$$Y(X) = \begin{cases} -X, & X \le 0 \\ X^2, & 0 < X \le 2 \\ 4, & X > 2 \end{cases}$$

Пройти діапазон циклом for або while, включивши праву межу відрізка. На кожній ітерації через if-else вибрати формулу та надрукувати таблицю X і Y(X). Якщо математична операція недопустима, замість значення вивести [Не визначено]. У варіанті 1 ділення й добування кореня відсутні.

1.2 Завдання 2

Розробити task2.c для наближеного обчислення e^x за рядом:

$$e^X = 1 + X + \frac{X^2}{2!} + \frac{X^3}{3!} + \cdots = \sum_{n=0}^{\infty} \frac{X^n}{n!}$$

Зчитати X та ε > 0 і перевірити область збіжності. Обчислювати кожний доданок виключно через попередній. Накопичувати суму циклом while до $|a_n| < \varepsilon$, контролюючи модуль функцією fabs. Передбачити ліміт 100 000 ітерацій із примусовим break. Вивести суму, exp(X), абсолютну похибку й кількість ітерацій; додатково обчислити відносну похибку.

2 Математичний аналіз та рекурентні співвідношення

2.1 Область допустимих значень та неперервність

Область допустимих значень функції із завдання 1 - всі дійсні числа: D(Y) = $\mathbb{R}$. Кожна гілка визначена на своєму інтервалі; ділення на нуль та коренів із від'ємних чисел у виразах немає.

Перевіряю межі гілок. При X → 0 зліва −X → 0, а справа X^2 → 0; Y(0) = 0. При X → 2 зліва X^2 → 4, а справа стала функція має значення 4; Y(2) = 4. Отже, функція неперервна на всій числовій осі. Точки X = 0 і X = 2 є лише межами вибору формул; точок розриву немає.

2.2 Виведення формули переходу для ряду

Загальний член та початковий доданок:

$$a_n = \frac{X^n}{n!},\ a_0 = 1$$

Для n ≥ 1 і X ≠ 0 знаходжу відношення сусідніх членів:

$$R(n, X) = \frac{a_n}{a_{n-1}} = \frac{X^n}{n!} \cdot \frac{(n-1)!}{X^{n-1}}$$

Оскільки n! = n · (n − 1)!, скорочую факторіали та степені:

$$R(n, X) = \frac{X^n}{X^{n-1}} \cdot \frac{(n-1)!}{n \cdot (n-1)!} = \frac{X}{n}$$

Остаточно отримую формулу для програмної реалізації:

$$a_n = a_{n-1} \cdot \frac{X}{n}$$

При X = 0 відношення сусідніх членів не завжди визначене, але рекурентна формула правильна: a_0 = 1, а всі наступні доданки дорівнюють нулю.

2.3 Область збіжності

Для фіксованого X ≠ 0 за ознакою Даламбера:

$$\frac{|a_n|}{|a_{n-1}|} = \frac{|X|}{n} \to 0 < 1 \quad (n \to \infty)$$

При X = 0 ряд також збігається до 1. Отже, область збіжності - $-\infty < X < +\infty$. Будь-який скінченний дійсний аргумент допустимий математично; машинні обмеження типу double перевіряю окремо.

3 Опис алгоритмів та логічних рішень

3.1 Алгоритм табулювання

Зчитую три значення через scanf й перевіряю, що функція успішно перетворила три числа. isfinite відхиляє NaN та нескінченності. При Xstart ≥ Xend або dX ≤ 0 програма завершується з кодом 1 до початку обчислень.

У циклі for починаю з Xstart. Якщо X ≤ 0, обчислюю −X; інакше при X ≤ 2 обчислюю X · X; для решти значень повертаю 4. Результат друкую з фіксованою шириною колонок. Перевірка isfinite перед виведенням дозволяє позначити нечисловий результат як [Не визначено].

Наступну точку знаходжу додаванням dX. Якщо вона перевищує Xend, окремо друкую праву межу й завершую цикл. Тому для відрізка [0; 1] із кроком 0,3 отримую точки 0; 0,3; 0,6; 0,9; 1. Останній інтервал може бути коротшим за dX.

Для захисту від зациклення перевіряю next > x: надто малий крок може не змінити double через округлення. Попередня оцінка числа кроків та захисний лічильник обмежують обсяг таблиці до 100 000 рядків.

3.2 Алгоритм підсумовування ряду

Зчитую X і ε, перевіряю число перетворень scanf, скінченність значень та ε > 0. Еталон exp(X) обчислюю один раз перед циклом. Якщо він нескінченний або округлився до нуля, повідомляю про вихід за діапазон double й повертаю код 1.

Початкові значення: term = 1, sum = 1, n = 0. Поки fabs(term) ≥ ε, збільшую n, обчислюю term *= X/n і додаю доданок до суми. Перший доданок, модуль якого менший за ε, включаю до суми; на наступній перевірці цикл завершується. n означає кількість рекурентних переходів, а кількість доданків у сумі дорівнює n + 1.

Перед переходом перевіряю ліміт 100 000 ітерацій. У разі його досягнення друкую попередження та виконую break. Після переходу перевіряю скінченність доданка й нової суми. Переповнення або вичерпання ліміту завершують програму з кодом 2.

Умова $|a_n| < \varepsilon$ є вимогою до останнього доданка, а не універсальною гарантією похибки суми. Тому після завершення порівнюю результат з exp(X). Для великих від'ємних X взаємне скорочення різнознакових доданків може знизити точність; при X < −20 програма виводить попередження.

4 Лістинги вихідного коду

4.1 Повний лістинг task1.c

```
#include <stdio.h>
#include <math.h>

#define MAX_ROWS 100000U

static void print_row(double x)
{
   double y;
   if (x <= 0.0) {
      y = -x;
   } else if (x <= 2.0) {
      y = x * x;
   } else {
      y = 4.0;
   }
   if (!isfinite(y)) {
      printf("%12.6g | [Не визначено]\n", x);
   } else {
      printf("%12.6g | %12.6f\n", x, y);
   }
}

int main(void)
{
   double start, end, dx;
   printf("Enter X_start X_end dX: ");
   if (scanf("%lf %lf %lf", &start, &end, &dx) != 3) {
      fprintf(stderr, "Error: enter three numbers.\n");
      return 1;
   }
   if (!isfinite(start) || !isfinite(end) ||
      !isfinite(dx) || start >= end || dx <= 0.0) {
      fprintf(stderr, "Error: finite values, start < end, dX > 0.\n");
      return 1;
   }
```

4.1 Лістинг task1.c продовження

```
    long double steps = ((long double)end - start) / dx;
    if (steps > MAX_ROWS - 2U) {
        fprintf(stderr, "Error: too many rows; increase dX.\n");
        return 1;
    }
    printf("%12s | %12s\n", "X", "Y(X)");
    unsigned int rows = 0;
    for (double x = start; x <= end; ) {
        if (++rows > MAX_ROWS) {
            fprintf(stderr, "Error: row limit reached.\n");
            return 1;
        }
        print_row(x);
        if (x == end) {
            break;
        }
        double next = x + dx;
        if (next <= x) {
            fprintf(stderr, "Error: dX too small for double.\n");
            return 1;
        }
        if (!isfinite(next) || next > end) {
            print_row(end);  /* Include the right endpoint. */
            break;
        }
        x = next;
    }
    return 0;
}
```

Компіляція task1.c

```
gcc -Wall -Wextra task1.c -o task1 -lm
./task1
```

Ключі -Wall та -Wextra вмикають діагностування попереджень, а -lm підключає математичну бібліотеку. Вихідний файл компілюється без попереджень.

4.2 Повний лістинг task2.c

```
#include <stdio.h>
#include <math.h>

#define MAX_ITER 100000U

int main(void)
{
   double x, eps;
   printf("Enter X epsilon: ");
   if (scanf("%lf %lf", &x, &eps) != 2) {
      fprintf(stderr, "Error: enter two numbers.\n");
      return 1;
   }
   /* The series for exp(x) converges for every real x. */
   if (!isfinite(x) || !isfinite(eps) || eps <= 0.0) {
      fprintf(stderr, "Error: finite X and epsilon > 0 required.\n");
      return 1;
   }
   double reference = exp(x);
   if (!isfinite(reference) || reference == 0.0) {
      fprintf(stderr, "Error: exp(X) is outside double range.\n");
      return 1;
   }
   double term = 1.0;
   double sum = term;
   unsigned int n = 0;
   int failed = 0;
   /* a_0 = 1; a_n = a_(n-1) * x / n. */
   while (fabs(term) >= eps) {
      if (n >= MAX_ITER) {
         fprintf(stderr, "Warning: iteration limit reached.\n");
         failed = 1;
         break;
      }
      ++n;
```

4.2 Лістинг task2.c продовження

```
        term *= x / (double)n;
        double next_sum = sum + term;
        if (!isfinite(term) || !isfinite(next_sum)) {
            fprintf(stderr, "Error: intermediate overflow.\n");
            failed = 1;
            break;
        }
        sum = next_sum;
    }
    if (failed) {
        return 2;
    }
    double abs_error = fabs(sum - reference);
    double rel_error = abs_error / fabs(reference);
    printf("Series sum      = %.12g\n", sum);
    printf("exp(X)          = %.12g\n", reference);
    printf("Absolute error = %.12g\n", abs_error);
    printf("Relative error = %.12g\n", rel_error);
    printf("Iterations      = %u\n", n);
    printf("Last term       = %.12g\n", term);
    if (x < -20.0) {
        printf("Warning: cancellation may reduce accuracy.\n");
    }
    return 0;
}
```

Компіляція task2.c

```
gcc -Wall -Wextra task2.c -o task2 -lm
./task2
```

Вихідний файл компілюється без попереджень. Для першого запуску використовую $X = 1$, $\varepsilon = 10^{-6}$. У терміналі отримую:

```
Series sum      = 2.71828180115
exp(X)          = 2.71828182846
Absolute error = 2.73126605776e-08
Relative error = 1.00477663102e-08
Iterations      = 10
Last term       = 2.7557319224e-07
```

5 Результати тестування та верифікація

5.1 Табулювання заданої функції

Основний тест: Xstart = −2, Xend = 3, dX = 0,5. Таблиця містить усі три гілки та обидві межові точки.

X	Y(X)
-2.00	2.000000
-1.50	1.500000
-1.00	1.000000
-0.50	0.500000
0.00	0.000000
0.50	0.250000
1.00	1.000000
1.50	2.250000
2.00	4.000000
2.50	4.000000
3.00	4.000000

Для X = 0 консоль може показувати −0.000000 через знак нуля в double; чисельно це 0. При X = 2 використовується друга гілка, яка повертає 4; праворуч від 2 стала гілка теж повертає 4.

```
hallt@DragonWing ~/p/p/lab02 (main)> ./task1
Enter X_start X_end dX: -2 3 0.5
           X |          Y(X)
          -2 |     2.000000
        -1.5 |     1.500000
          -1 |     1.000000
        -0.5 |     0.500000
           0 |    -0.000000
         0.5 |     0.250000
           1 |     1.000000
         1.5 |     2.250000
           2 |     4.000000
         2.5 |     4.000000
           3 |     4.000000
hallt@DragonWing ~/p/p/lab02 (main)>
```

Рисунок 1 - Табулювання функції на відрізку [−2; 3] із кроком 0,5

Додаткові перевірки меж гілок

X	Y(X)
-0.01	0.010000
0.01	0.000100

X	Y(X)
1.99	3.960100
2.01	4.000000

При Xstart = 0, Xend = 1, dX = 0,3 програма додатково друкує X = 1. Це підтверджує включення правої межі, коли довжина відрізка не кратна кроку.

5.2 Збіжність ряду та оцінювання похибки

Обчислення виконано для X = −1, 1 і 3 при ε = 10^{-2}, 10^{-4} та 10^{-6}. Еталонні значення отримано через exp(X) із <math.h>. Похибки визначаю так:

$$\Delta = |S - exp(X)|, \quad \delta = \frac{|S - exp(X)|}{|exp(X)|}$$

X	ε	Сума S	exp(X)	Δ	δ	n
-1	1e-02	0.3666666667	0.3678794412	1.213e-03	3.297e-03	5
-1	1e-04	0.3678819444	0.3678794412	2.503e-06	6.805e-06	8
-1	1e-06	0.3678794643	0.3678794412	2.311e-08	6.283e-08	10
1	1e-02	2.7166666667	2.7182818285	1.615e-03	5.942e-04	5
1	1e-04	2.7182787698	2.7182818285	3.059e-06	1.125e-06	8
1	1e-06	2.7182818011	2.7182818285	2.731e-08	1.005e-08	10
3	1e-02	20.0841030844	20.0855369232	1.434e-03	7.139e-05	11
3	1e-04	20.0855234581	20.0855369232	1.347e-05	6.704e-07	14
3	1e-06	20.0855368515	20.0855369232	7.174e-08	3.572e-09	17

Похибки обчислено за значеннями до округлення для друку. Відносна похибка δ подана як частка; для відсотків її треба помножити на 100.

При |X| = 1 зменшення ε з 10^{-2} до 10^{-6} збільшує кількість ітерацій з 5 до 10. При X = 3 потрібно відповідно 11 і 17 переходів. Отже, в цих тестах віддалення від точки розкладу збільшує число доданків, необхідних для виконання умови завершення.

Додатковий тест X = 0, ε = 10^{-6}: отримано суму 1, еталон 1, обидві похибки 0 та одну ітерацію. Перший рекурентний доданок дорівнює нулю.

```
X   epsilon   sum              exp(X)           abs_error    rel_error    n
-1  1e-2     0.366666666667   0.367879441171   0.00121      0.0033       5
-1  1e-4     0.367881944444   0.367879441171   2.5e-06      6.8e-06      8
-1  1e-6     0.367879464286   0.367879441171   2.31e-08     6.28e-08     10
 1  1e-2     2.71666666667    2.71828182846    0.00162      0.000594     5
 1  1e-4     2.71827876984    2.71828182846    3.06e-06     1.13e-06     8
 1  1e-6     2.71828180115    2.71828182846    2.73e-08     1e-08        10
 3  1e-2     20.0841030844    20.0855369232    0.00143      7.14e-05     11
 3  1e-4     20.0855234581    20.0855369232    1.35e-05     6.7e-07      14
 3  1e-6     20.0855368515    20.0855369232    7.17e-08     3.57e-09     17
hallt@DragonWing:~/projects/programming-sem1/lab02$
```

Рисунок 2 - Обчислення e^x для різних аргументів та значень точності

5.3 Перевірка некоректних вхідних даних

Програма	Вхідні дані	Реакція	Код
task1	-2 3 -0.5	Відхилено від'ємний крок	1
task1	-2 3 0	Відхилено нульовий крок	1
task1	3 -2 0.5	Відхилено зворотні межі	1
task1	2 2 0.5	Відхилено рівні межі	1
task1	abc 3 0.5	Помилка зчитування чисел	1
task2	1 0	Точність повинна бути додатною	1
task2	nan 1e-6	X повинен бути скінченним	1
task2	1000 1e-6	exp(X) поза діапазоном double	1

Код завершення перевіряю командою echo $? одразу після запуску програми. При від'ємному кроці отримую Error: finite values, start < end, dX > 0.

```
Error: finite values, start < end, dX > 0.
Enter X_start X_end dX: Exit code: 1
hallt@DragonWing:~/projects/programming-sem1/lab02$
```

Рисунок 3 - Відхилення від'ємного кроку в task1.c

При введенні X = 1 та ε = 0 отримую Error: finite X and epsilon > 0 required. Для NaN програма також завершується з кодом 1.

```
Error: finite X and epsilon > 0 required.
Enter X epsilon: Exit code: 1
hallt@DragonWing:~/projects/programming-sem1/lab02$
```

Рисунок 4 - Відхилення недопустимої точності в task2.c

Для ряду e^x немає скінченного дійсного X поза областю збіжності. Тому вимогу методички щодо такого тесту пояснюю властивістю варіанта 1. Тест X = 1000 перевіряє переповнення exp(X) у типі double, а не розбіжність ряду.

```
Error: exp(X) is outside double range.
Enter X epsilon: Exit code: 1
hallt@DragonWing:~/projects/programming-sem1/lab02$
```

Рисунок 5 - Перевірка машинного обмеження при X = 1000

6 Висновки

Для варіанта 1 реалізовано табулювання кусочно заданої функції та наближене обчислення експоненти. Вибір гілки функції виконується через if-else, табулювання - циклом for, накопичення ряду - циклом while. Захисні перевірки виявляють нечислове введення, некоректні межі, недодатні крок і точність, переповнення та перевищення лімітів.

Математичний аналіз показав, що кусочна функція визначена й неперервна на всій числовій осі. Тестування значень біля X = 0 і X = 2 підтвердило правильність умов вибору гілок. Програма включає праву межу відрізка навіть тоді, коли його довжина не кратна кроку.

Виведено формулу $a_n = a_{n-1} \cdot X/n$. Для кожного переходу потрібні ділення, множення та додавання до суми; степені й факторіали окремо не обчислюються. Для N переходів алгоритм потребує O(N) арифметичних операцій і O(1) додаткової пам’яті. Повторне обчислення факторіалів окремими циклами для всіх N членів вимагало б $O(N^2)$ множень.

Зменшення ε підвищило точність тестових результатів та збільшило кількість ітерацій. Для X = 1 і $\varepsilon = 10^{-6}$ отримано суму 2,71828180115 з абсолютною похибкою приблизно $2{,}73 \cdot 10^{-8}$ за 10 переходів. При X = 3 для тієї самої точності потрібно 17 переходів, що демонструє уповільнення збіжності при віддаленні від точки розкладу.

Рекурентне обчислення ефективніше за повторне пряме обчислення кожного доданка через степінь і факторіал. Це не означає, що навчальний алгоритм швидший за оптимізовану бібліотечну exp: вона використана як еталон для перевірки, а порівняльне вимірювання часу в цій роботі не проводилося.

Для великих від’ємних аргументів можливе значне зниження точності через взаємне скорочення доданків. Математична збіжність ряду для всіх дійсних X не усуває обмежень машинної арифметики. Мету роботи досягнуто: реалізовано розгалужені та циклічні алгоритми, використано рекурентні співвідношення, перевірено коректність введення й оцінено похибки.